

019: Soft-GiGPO: 基于模型既视感的 Step 软匹配设想

2026-06-10

摘要

本文草拟一个新的 GiGPO 扩展方向: 把当前基于 anchor observation 的硬文本匹配, 替换或增强为基于模型“既视感”的 soft semantic matching。核心思想是: 两个 step 不必完全文本相同才可以互相作为 step-level baseline; 只要它们在模型表示空间、文本 embedding 空间或 frozen actor hidden space 中足够相似, 就可以按连续权重参与对方的 relative advantage 估计。

这个方向暂命名为 **Soft-GiGPO / Semantic Step Matching**。它不改变当前 StepPPO-v3 的 value-head-only 定义, 也不默认引入 action-level PPO ratio 或新的 critic。第一版建议只替换 GiGPO 的 step relative advantage estimator, 保持 rollout、policy loss 和训练 recipe 不变。

1 背景: 当前 GiGPO 的硬匹配

当前 GiGPO 的关键设计是: 在同一个任务 uid 下, 把处在相同 anchor state 的 rollout step 放入同一个 step group, 然后在 group 内计算相对 step advantage。这个设计比只看 episode-level reward 更细, 因为它试图回答: 在“相同状态”下, 不同 response/action 的未来 return 谁更好。

代码路径大致是:

```
gigpo/core_gigpo.py
compute_gigpo_outcome_advantage(...)
    step_group_uids = build_step_group(anchor_obs, index, enable_similarity, similarity_thresh)
    step_advantages = step_norm_reward(step_rewards, response_mask, step_group_uids, ...)
```

当前实现仍然主要是硬分组: 默认 exact match; 可选 enable_similarity=True 时使用 SequenceMatcher 文本相似度和阈值做 hard clustering。一旦两个 step 没落入同一 cluster, 它们对彼此的 baseline/advantage 就完全没有贡献。

这种硬匹配在 WebShop 这样的文本环境中有明显限制: 页面 observation 可能只因为商品排序、空格、价格显示、历史上下文或轻微 wording 改变而无法 exact match; 反过来, 两个文本相似的页面也可能在任务语义上不同。于是 step-level signal 可能既稀疏又脆弱。

2 直觉: 模型的“既视感”

用户提出的“模型既视感”可以理解为: 当模型看到当前 step context 时, 它内部可能已经知道这个状态和 batch 内哪些历史或并行 rollout state 很像。这个像不像不一定等价于文本完全相同, 而可能包含:

- 当前商品搜索页面是否语义相近;
- 当前 observation 是否处在相似的搜索、筛选或详情页阶段;
- 当前 action space 是否类似;

- 当前任务进展是否类似；
- 当前 hidden representation 是否显示出相似的决策处境。

如果这种相似度可用，就可以把 GiGPO 的“同 anchor state 内相对比较”推广成“相似 state 之间的软相对比较”。一个 step 不再只和 exact-matched peers 比较，而是和一组相似 step 按权重比较。

3 方法定义

设一个 batch 中有 step 样本 $i = 1, \dots, N$ 。每个样本属于任务组 u_i ，有 anchor/context 表示 s_i ，step-level return 或 discounted step reward 为 R_i 。原始 GiGPO 会构造 hard group G_i ，然后计算：

$$A_i^{\text{hard}} = R_i - \frac{1}{|G_i|} \sum_{j \in G_i} R_j, \quad (1)$$

或在 `mean_std_norm` 下再除以 group std。

Soft-GiGPO 改为先定义相似表示和相似度：

$$z_i = f_{\text{match}}(s_i), \quad \text{sim}_{ij} = \cos(z_i, z_j). \quad (2)$$

其中 f_{match} 可以是 frozen embedding model、actor selected hidden state、value head 的中间表示，或一个单独训练的轻量 projection。

在同一 task uid 内计算 kernel 权重：

$$w_{ij} = \frac{\mathbf{1}[u_i = u_j] \mathbf{1}[i \neq j] \exp(\text{sim}_{ij}/\tau)}{\sum_{k: u_k = u_i, k \neq i} \exp(\text{sim}_{ik}/\tau) + \epsilon}. \quad (3)$$

这里 τ 是 temperature，控制 soft matching 的尖锐程度。若 τ 很小，方法接近 nearest-neighbor；若 τ 较大，则接近 uid 内平均 baseline。

soft baseline 为：

$$b_i^{\text{soft}} = \sum_{j: u_j = u_i, j \neq i} w_{ij} R_j. \quad (4)$$

soft step advantage 为：

$$A_i^{\text{soft}} = R_i - b_i^{\text{soft}}. \quad (5)$$

如果需要标准化，可以用加权方差：

$$\sigma_i^2 = \sum_j w_{ij} (R_j - b_i^{\text{soft}})^2, \quad \hat{A}_i^{\text{soft}} = \frac{R_i - b_i^{\text{soft}}}{\sqrt{\sigma_i^2 + \epsilon}}. \quad (6)$$

为了避免自身 reward 泄漏，默认应使用 leave-one-out，即 $w_{ii} = 0$ 。

4 和 GiGPO 的关系

Soft-GiGPO 不是推翻 GiGPO，而是把 hard anchor group 泛化成 soft anchor neighborhood。原始 GiGPO 可以看成一种特殊 kernel：

$$w_{ij}^{\text{hard}} \propto \mathbf{1}[g_i = g_j] \mathbf{1}[i \neq j]. \quad (7)$$

Soft-GiGPO 则把 indicator 变成连续 similarity kernel：

$$w_{ij}^{\text{soft}} \propto K(s_i, s_j). \quad (8)$$

因此它保留 GiGPO 的核心精神：仍然是 group-relative / state-relative advantage；区别只是“状态相同”的定义从离散硬匹配变成连续软匹配。

5 “模型既视感”的实现选择

5.1 文本 embedding 版

对 anchor observation 或 pre-action context 做 embedding:

$$z_i = \text{Embed}(\text{text}(s_i)). \quad (9)$$

优点是实现快，不依赖 actor forward 额外改动；缺点是外部 embedding 不一定理解 WebShop action semantics，也可能引入额外模型和 runtime。

5.2 Actor hidden state 版

使用 actor 在准备生成 action 前的 selected hidden state:

$$z_i = h_{\text{pre-action-last-context-token}, i}. \quad (10)$$

这和当前 v3 value head 使用的 state representation 接近。优点是“既视感”来自同一个 policy model，可能更贴近策略决策；缺点是 representation 非平稳，训练过程中会漂移。

5.3 Frozen actor snapshot 版

为避免 representation non-stationarity，可以用 frozen checkpoint 或 reference actor 计算 z_i ，例如初始 SFT actor、GiGPO baseline checkpoint，或周期性冻结的 target encoder。优点是相似度稳定；缺点是可能逐渐落后于当前 policy 的状态分布。

5.4 Value-aware 版

如果 v3 value head 已证明 selected hidden state 有 step return signal，可以用 value prediction 辅助相似度:

$$\text{sim}_{ij} = \alpha \cos(z_i, z_j) - \beta |V(s_i) - V(s_j)|. \quad (11)$$

直觉是：两个状态不仅文本或 representation 相似，而且预测 return 也相近，才更适合作为 baseline peer。这个版本风险较高，因为它会把 learned value 的 bias 引入 advantage estimator，应作为后续 ablation。

6 推荐第一版：Soft-GiGPO-v0

第一版建议只替换 GiGPO 的 step group / step advantage 计算，不改 policy loss，不接入 StepPPO-v3 value head，不改变 rollout recipe。

```
algorithm:
  adv_estimator: gigpo
  gigpo:
    step_advantage_w: 1.0
    mode: mean_norm
    soft_matching:
      enable: true
      encoder: text_embedding          # text_embedding / actor_hidden / frozen_actor
      scope: same_uid                 # only match within the same task uid
      kernel: cosine_softmax
      temperature: 0.1
      top_k: 8
```

```

min_neighbors: 2
leave_one_out: true
fallback: hard_or_uid_mean
detach_embeddings: true

```

计算流程:

1. 对每个 step 构造匹配表示 z_i 。
2. 在同一 uid 内计算 pairwise similarity。
3. 取 top-k neighbors, 做 softmax 权重。
4. 用 weighted leave-one-out return baseline 得到 A_i^{soft} 。
5. 把 scalar step advantage broadcast 到 response tokens, 保持原 GiGPO actor update 不变。

7 关键设计选择

匹配范围限制在同一任务 uid 内。 不同 WebShop task 的目标商品不同, 即使页面文本相似, reward 语义也可能完全不同。第一版应只允许同一 uid 内的 rollout 相互匹配。

使用 top-k 而不是全连接 dense kernel。 全连接 softmax 可能把大量弱相关样本纳入 baseline, 导致 advantage 过度平滑。top-k 能保留“相似 neighborhood”的直觉, 并降低计算开销。

默认 leave-one-out。 如果把自己放进 baseline, 尤其当 top-k 很小或 similarity 很尖锐时, 会把 R_i 泄漏进 b_i , 使 advantage 被人为压小。因此默认 $w_{ii} = 0$ 。

fallback 很重要。 如果某个 uid 内有效 neighbor 太少, 或相似度全都很低, 应该 fallback 到原始 hard GiGPO group、uid 内 mean baseline, 或直接把 step advantage 置零, 只保留 episode advantage。第一版建议 fallback=hard_or_uid_mean。

8 诊断指标

Soft matching 是否有用, 不能只看 validation score。必须先确认 matching 质量和 advantage 行为:

```

gigpo_soft/neighbor_count_mean
gigpo_soft/effective_neighbors_mean
gigpo_soft/top1_similarity_mean
gigpo_soft/topk_similarity_mean
gigpo_soft/weight_entropy_mean
gigpo_soft/fallback_ratio
gigpo_soft/soft_baseline_mean
gigpo_soft/soft_adv_mean
gigpo_soft/soft_adv_std
gigpo_soft/hard_soft_adv_corr
gigpo_soft/group_size_equiv_mean

```

其中 effective neighbors 可以定义为:

$$N_{\text{eff},i} = \frac{1}{\sum_j w_{ij}^2}. \quad (12)$$

它能判断 soft kernel 是退化成单邻居 hard match，还是退化成过宽平均。

还应做 offline diagnostic: hard group size vs soft effective neighbor size; hard advantage 与 soft advantage 的相关性; top-k neighbors 的 observation 文本人工抽样; soft similarity 与 reward gap 的关系; soft matching 是否降低 singleton 比例; soft advantage 是否减少极端 outlier。

9 可能收益与主要风险

9.1 可能收益

1. 缓解 exact-match 稀疏性: 相似但不完全相同的 states 可以互相提供 baseline。
2. 更细粒度 credit assignment: 不必等到完全同一 anchor observation 才比较 response/action。
3. 提高 sample efficiency: 同一 batch 内更多 step 能参与 step-relative normalization。
4. 兼容当前 GiGPO pipeline: 第一版只替换 step advantage, 不改 actor loss、ratio、rollout。
5. 为 value head 提供诊断桥梁: 如果 actor hidden state 的 soft matching 有用, 说明 representation 中已有可用 state similarity signal。

9.2 主要风险

1. 错误匹配引入 bias: 语义相似不等于决策等价, 错误 neighbors 会污染 baseline。
2. 过度平滑 advantage: soft baseline 过宽会把有用差异抹掉。
3. representation 漂移: 若用当前 actor hidden state, similarity 空间会随训练变化。
4. reward leakage: 不做 leave-one-out 会低估 advantage。
5. 额外计算开销: pairwise similarity 需要 uid 内 top-k 和缓存。
6. 与 v3 value head 混淆: 当前 StepPPO-v3 主线仍应保持 value-head-only; soft matching 应作为新的 GiGPO ablation 或未来版本。

10 实验计划

10.1 Offline matching study

先不训练, 只用已有 GiGPO rollout/log dump 或新增少量 rollout dump:

1. 对每个 batch 构造 hard groups 和 soft neighborhoods。
2. 统计 singleton hard groups 被 soft matching 扩展后的 effective neighbor 数。
3. 人工检查 top-k neighbors 是否语义合理。
4. 比较 hard advantage 与 soft advantage 的均值、方差、极端值和相关性。
5. 检查 soft similarity 是否和 reward gap / future return gap 有负相关。

如果 soft neighbors 看起来不合理, 不应直接进入 full training。

10.2 2GPU smoke

目标不是追分，而是看 estimator 是否稳定：valid action ratio 不快速崩；response clip ratio 不快速升高；soft fallback ratio 不长期接近 1；effective neighbors 不塌缩到 1，也不过宽接近 uid group size；policy loss、KL、grad norm 不异常。

10.3 4GPU seed 2026 full run

只和已有 GiGPO seed 2026 对齐：同 batch size、rollout group size、test freq、GPU 数。比较 validation task score / success、valid action ratio、response length / clip ratio、runtime、hard-soft advantage diagnostics。

10.4 Ablation 顺序

实验	改动	目的
GiGPO hard	当前 baseline	对照
Soft-GiGPO text-emb top-k	外部/frozen text embedding	验证软匹配基本收益
Soft-GiGPO actor-hidden frozen	frozen actor representation	验证模型既视感是否优于文本 embedding
Soft-GiGPO actor-hidden online	当前 actor hidden, detach	验证 online representation 是否可用
Soft-GiGPO hard+soft mix	$A = A_{hard} + \lambda A_{soft}$	降低纯 soft 风险

第一轮不建议同时加入 value head、action-level PPO ratio 或 StepPPO-v3 其它改动。

11 代码实现草图

可以新增：

```
def compute_soft_step_advantage(
    step_rewards: torch.Tensor,
    response_mask: torch.Tensor,
    uid: np.ndarray,
    match_embeddings: torch.Tensor,
    temperature: float = 0.1,
    top_k: int = 8,
    remove_std: bool = True,
    leave_one_out: bool = True,
    epsilon: float = 1e-6,
):
    # 1. split rows by uid
    # 2. cosine similarity within each uid
    # 3. mask self, take top-k
    # 4. softmax(sim / temperature)
    # 5. weighted baseline and optional weighted std
    # 6. broadcast scalar advantage to response tokens
    return step_advantages, diagnostics
```

并在 compute_gigpo_outcome_advantage 中加开关：

```
if soft_matching.enable:
    step_advantages, soft_metrics = compute_soft_step_advantage(...)
else:
```

```
step_group_uids = build_step_group(...)
step_advantages = step_norm_reward(...)
```

如果第一版使用 text embedding, embedding 可以先在 trainer driver 侧生成并放入 `DataProto.batch["step_match_embeddings"]`。如果使用 actor hidden state, 则可以复用 v3 value head 的 selected-state extraction 思路, 但第一版应只 dump/compute embedding, 不训练 value head。

12 与 StepPPO-v3 的边界

这份设想不改变当前 StepPPO-v3 定义。当前 v3 仍然是：

```
GiGPO policy training + actor-side shared value head
```

Soft-GiGPO 是另一个方向：改 GiGPO 的 step relative advantage estimator, 把 hard anchor matching 换成 soft semantic matching。它可以作为 GiGPO ablation、后续 GiGPO-v2 / Soft-GiGPO, 或将来 StepPPO-v4 的一个可能组件。但在当前阶段, 不应把 soft matching、value head、action-level PPO ratio 一次性合并, 否则很难定位收益或退化来源。

13 初步结论

这个想法值得做, 而且比直接把 value head 接入 policy advantage 更稳健：它仍然保留 GiGPO 的 relative baseline 思路, 只是把“相同 state”的硬文本定义放宽成“相似 state”的软语义定义。

建议下一步先做 offline matching study：用已有 GiGPO 或 v3 rollout batch, 比较 hard group 与 soft neighborhood 的质量。如果模型既视感能稳定找到语义合理、return 分布相近的 peers, 再进入 2GPU smoke 和 4GPU full run。